Level27 deployment

How production is provisioned on Level27, what each component does, how a deploy actually runs end to end, and how load is tested safely against the live site.

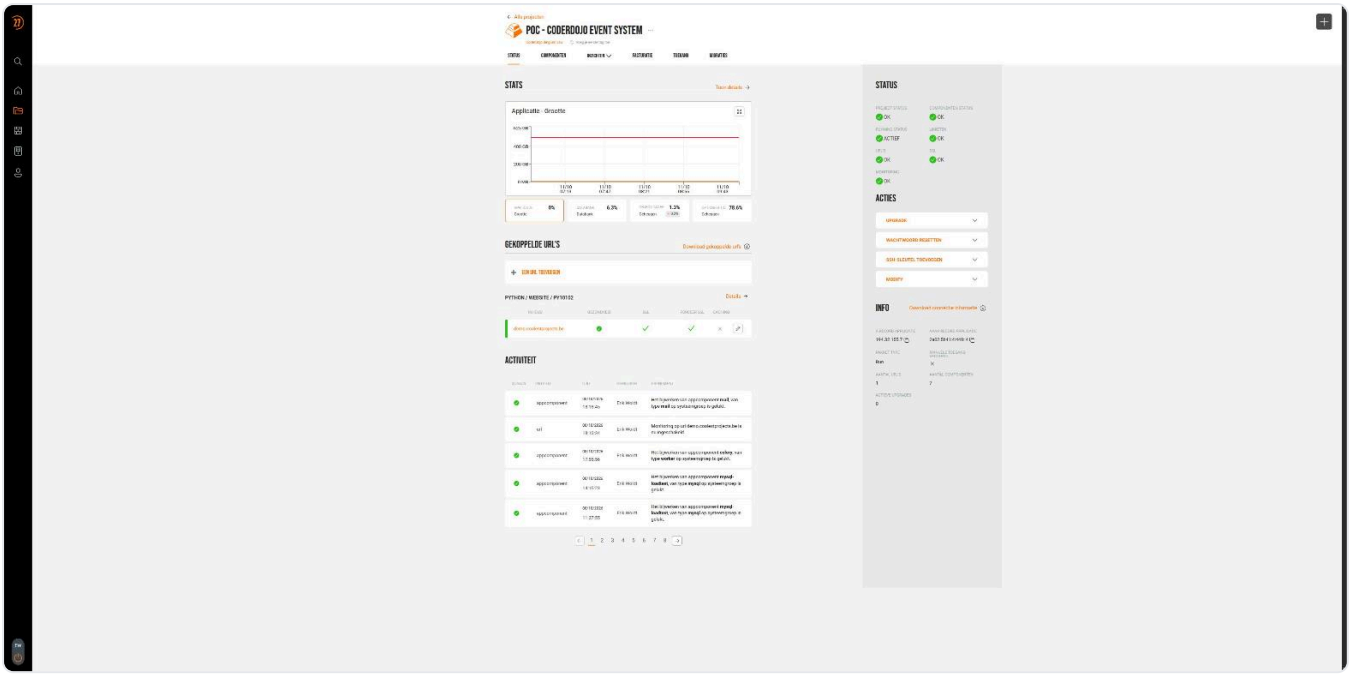
Contents

1. What's provisioned
2. The components, one by one
3. Networking, TLS and the domain
4. The deploy procedure
5. The two Celery workers on Level27
6. Load testing in production
7. Known constraints and gotchas

Generated from the repository and the Level27 panel, 11 October 2026 · user-journeys/scripts/build_level27_deployment.py · full prose in LEVEL27_DEPLOYMENT.md

1 What's provisioned

Production runs as one Level27 project (“POC - Coderdojo Event System”, project id `22212`), package type **Run**, with **7 components** reached only through Level27's own reverse proxy for one linked URL, `demo.coolestprojects.be`. There is no root access to the underlying server: everything here is managed either through the Level27 panel, or over SSH as the unprivileged account user `py10102`. This page is a snapshot of the panel on 11 October 2026 — the panel is the source of truth, this a guided tour of it.

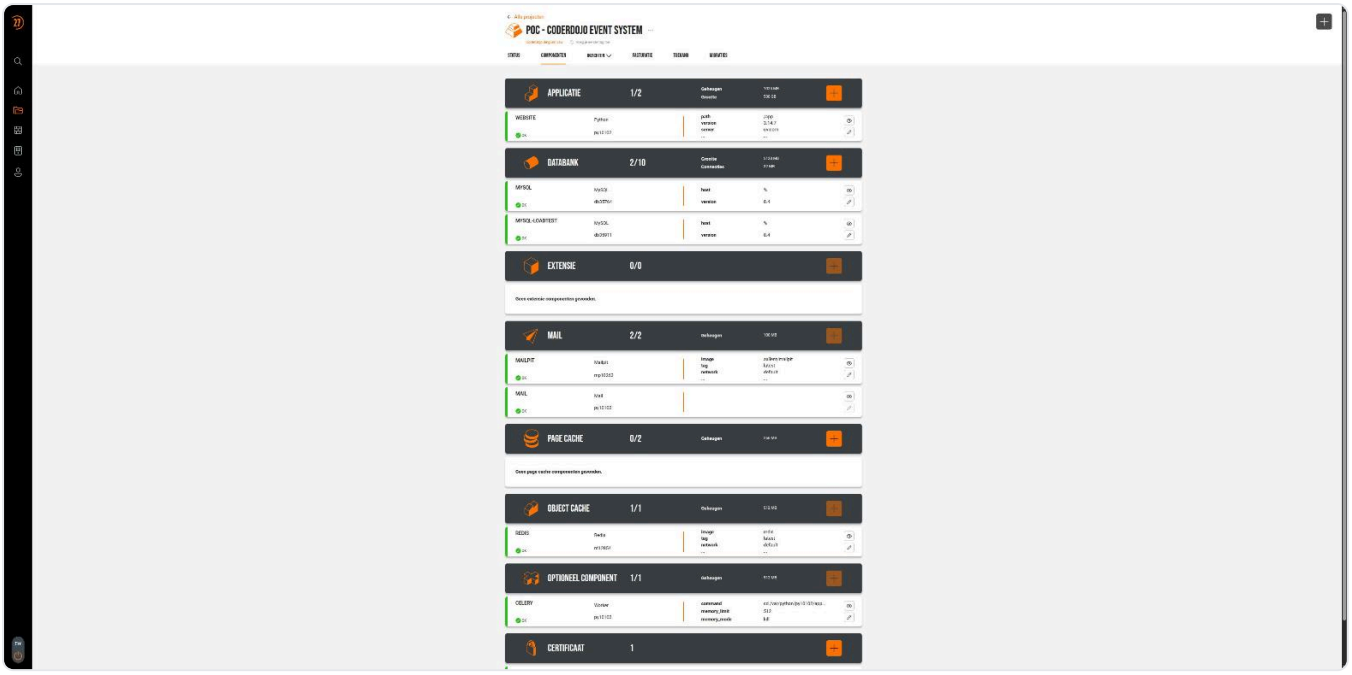


The project's own status page: project, component, running, limits, URL and SSL status all OK; the linked URL demo.coolestprojects.be with health, SSL and forced SSL green, caching off.

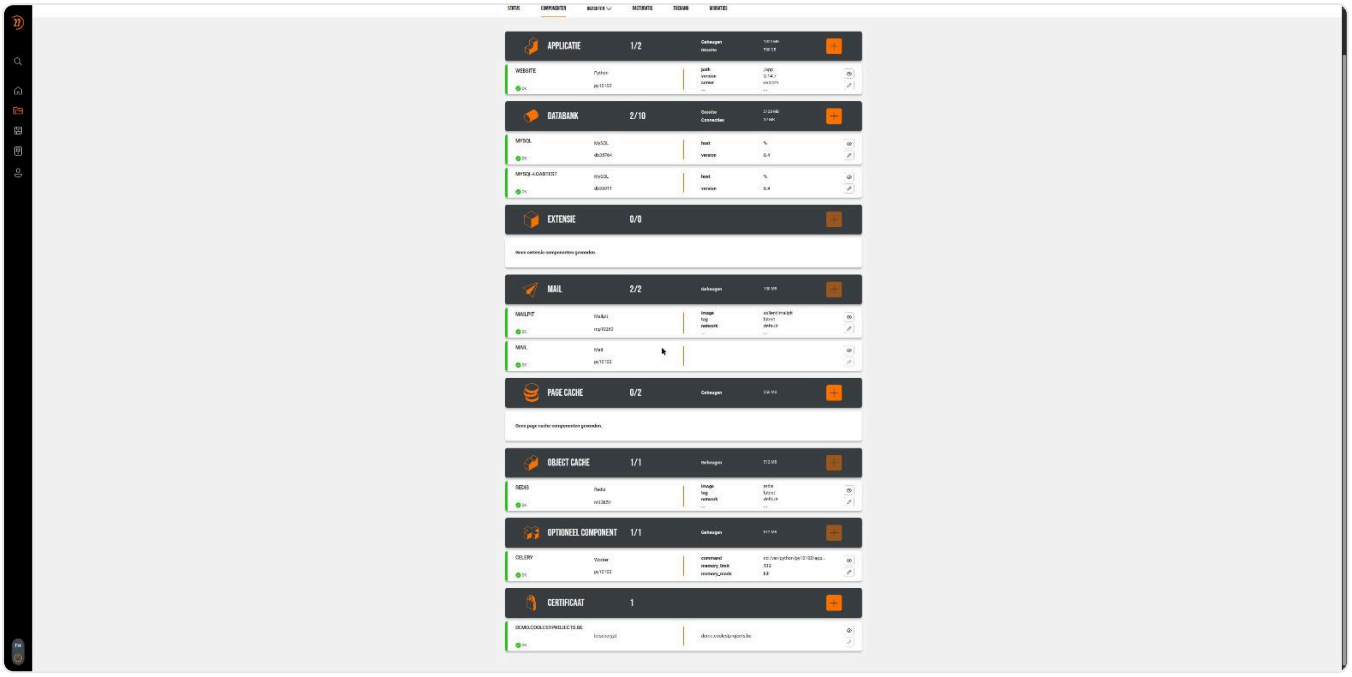
The 7 components

Group	Component	Type / image	Level27 id	Purpose
Applicatie	`website`	Python 3.14.7, uvicorn ASGI, `main:app`	`py10102`	The Django app
Databank	`mysql`	MySQL 8.4	`db03764`	Production database
Databank	`mysql-loadtest`	MySQL 8.4	`db05411`	Disposable database for production load tests
Mail	`mailpit`	`axllent/mailpit:latest`	`mp10263`	Catches mail only during a load test or demo switch
Mail	`mail`	Level27's outbound relay	`py10102`	The real SMTP path production mail goes out through
Object cache	`redis`	`redis:latest`	`rd12854`	Cache (db 0), Channels (db 1) and Celery broker (db 2)
Optioneel component	`celery`	“Worker” type, two commands	`py10102`	Both Celery workers

Extensie and Page cache groups are present but empty (0 components each) — by design: the site caches *data* in Redis, never whole pages (`CLAUDE.md` , “Caching”), so there is deliberately no Level27 page-cache component.



Components tab, top half: Applicatie, Databank, Extensie, Mail.



Components tab, bottom half: Mail, Page cache, Object cache, Optioneel component, Certificaat.

Architectural choices the panel confirms

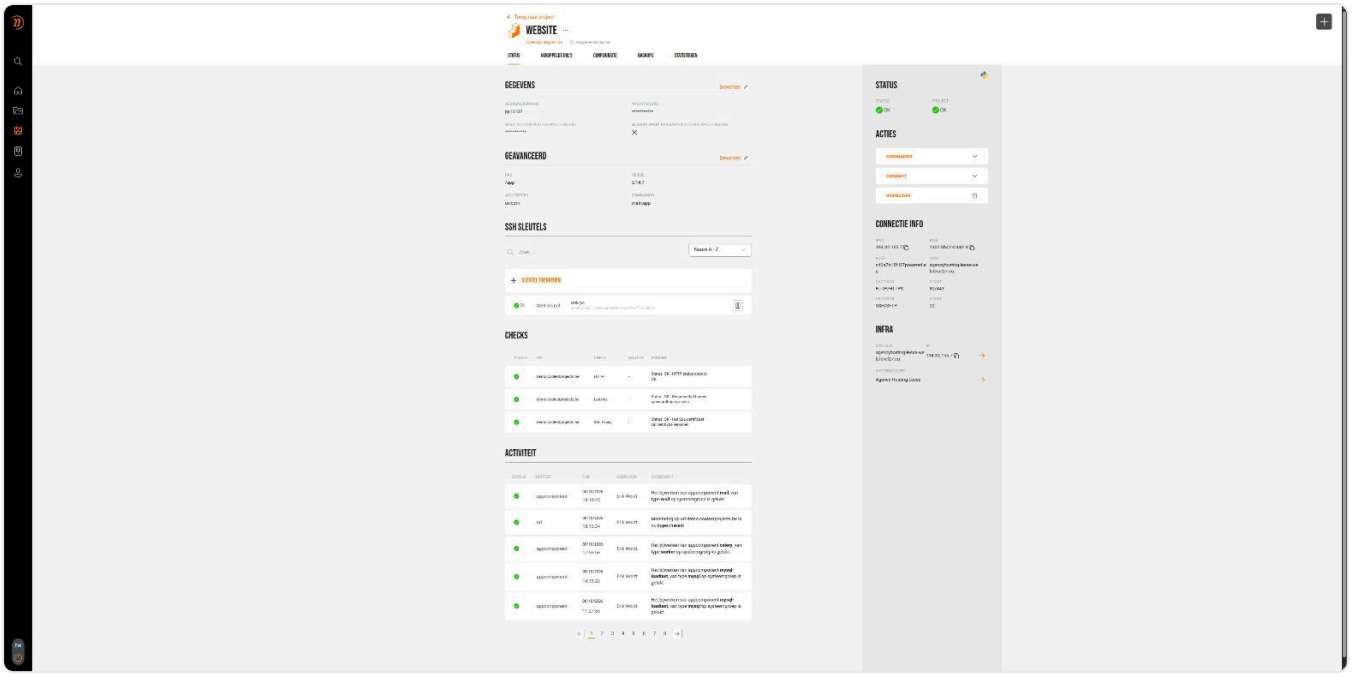
- **The database runs on its own dedicated host**, `agencyhosting-lesse-mysql8.level27.eu` (internal IP `172.21.155.6`), separate from the web app's host, `agencyhosting-lesse-web.level27.eu` (`194.32.155.7`).
- **The “Mail” app component and the “website” component share one Level27 app id** (`py10102`): Level27's outbound relay is attached to the app, not a separate container — matching

`EMAIL_HOST=localhost` in production's `.env`.

- **Caching is off at the proxy** for the linked URL — intentional: every page carries its own CSP nonce, CSRF token and per-account nav, so only the data behind it is cached, in the app's own Redis.
- **Forced SSL is on at the proxy**, independent of Django's own `SECURE_SSL_REDIRECT` (still `false` pending confirmation Level27's proxy always sets `X-Forwarded-Proto`): the proxy already refuses plain HTTP for this URL, ahead of whatever Django's own setting decides.
- **A second, disposable MySQL database** (`mysql-loadtest`) sits provisioned and idle most of the time, used only for a production load test.

2 The components, one by one

`website` (the Django app)



The website component's own detail page: credentials, path, SSH keys, checks, connection info, activity log.

- **Credentials panel**: username `py10102`, password masked (Level27-managed).
- **Advanced panel**: path `/app`, Python `3.14.7`, ASGI server `uvicorn`, command `main:app` — `main.py` re-exports `website.asgi.application` as `app`, and gunicorn runs `gunicorn -k uvicorn.workers.UvicornWorker main:app`.
- **SSH keys**: one key registered, used by `scripts/deploy.sh`.
- **Checks**: HTTP, latency and SSL expiry against `demo.coolestprojects.be`, all green.
- **Connection info**: IPv4 `194.32.155.7`, IPv6 `2a02:5b41:4:440::4`, FQDN `c40a7b15f.l27powered.eu`, HTTP/HTTPS 80/443, SSH/SFTP 22.
- **Activity log**: every panel change is attributed and timestamped — the authoritative record of who changed what in the panel, alongside `MAINTENANCE.md`'s security log and the application's own audit log.

`mysql` and `mysql-loadtest`

Both MySQL 8.4, host wildcard (`%`), on the same dedicated database host. `mysql` is the real production database; `mysql-loadtest` exists only so a load test can point the live app at throwaway data without touching real families' bookings.

`mailpit` and `mail`

`mail` (Level27's own relay) is what real production mail goes out through, all the time. `mailpit` catches mail only when the app is deliberately pointed at it during a production load test, so a test never sends real mail to real families.

`redis` (object cache)

One Redis component serves three logically separate databases — `CACHES` (db 0), Channels (db 1) and the Celery broker (db 2). Confirmed over SSH: Redis 8.10.1, `maxmemory` 512 MB, policy `allkeys-lru`, not yet the `volatile-lru` the caching design assumes.

`celery` (the optional “Worker” component)

One component, memory limit 512 MiB, memory mode **kill**, running two separate commands — see “The two Celery workers on Level27”.

Certificate

One certificate, `demo.coolestprojects.be`, issued via Let's Encrypt (auto-renewing), bound to the website application.

3 Networking, TLS and the domain

- The public entry point is `demo.coolestprojects.be`, resolving to the web app's IPv4/IPv6 and served, TLS included, by Level27's own reverse proxy — not anything in this repository. Apache serves `/static/` and `/media/` directly from disk under the account's home directory, and proxies everything else to the app's Unix socket, `/var/run/socket/py10102.socket`.
- SSH is the only other path in: `py10102@c40a7b15f.l27powered.eu`, port 22, key-based only, no root access.
- The database host is separate from the web host on Level27's internal network.
- Forced SSL is on for the linked URL; Django's own `SECURE_SSL_REDIRECT` is still off pending confirmation Level27's proxy always sets `X-Forwarded-Proto` — the proxy's own forced-SSL setting is a separate, already-active safety net ahead of that.
- The proxy's maximum request body size, and how it connects to the app's socket (new connection per request, or a kept-open pool), are both still unconfirmed with Level27.

4 The deploy procedure

Production is deployed with one script, `scripts/deploy.sh` (`--check` for a read-only preflight). Both talk to the server only over the same SSH connection the website component's registered key authorises.

1. **Bundles the working tree**, excluding development-only paths.
2. **Installs `requirements.txt`** into the exact pyenv environment gunicorn runs from (`~/.pyenv/versions/py10102-3.14.7`) — not a separate `~/app/.venv`.
3. **Runs `manage.py check` before touching the live app** — the main safety gate.
4. **Rsyncs** the new code into `~/app`, keeping `.env`, `media/`, `private_media/` and `staticfiles/` untouched.
5. **Migrates** the database.
6. **Runs `collectstatic`** into `~/app/staticfiles`, which Apache serves at `/static/`.
7. **Puts the new code live**: a graceful `HUP` to gunicorn, or a systemd stop/start for daphne (about 30 s down) — `deploy.sh` detects which is actually running.
8. **Smoke-tests over the Unix socket** directly, bypassing the proxy.
9. **Restarts both Celery workers with a warm shutdown**: `SIGTERM`, lets the in-flight task finish, Level27's worker component restarts the process on the new code, then the script pings both.
10. **`chmod o+x "$HOME"` on every deploy** — guards against a concrete gotcha: switching the app server in the panel resets the home directory to `750`, stripping the bit Apache needs to traverse into `~/app/staticfiles` / `media`. Caused a real outage (every visitor 403 on static/media for 7+ minutes) on 8 October 2026 when a panel-only change reset the permission outside of any code deploy. **Not fully closed**: a panel change that isn't a code deploy still needs this run by hand.

> **Why.** Production env vars live only on the server, in ~/app/.env, never in this repository. manage.py check before touching the live app is what stops a broken migration or missing setting from ever reaching production, and the warm Celery shutdown is the only lever available since there's no systemd access to create units of our own on this account.

5 The two Celery workers on Level27

Both workers run as one Level27 “Optioneel component” (type Worker) — the mechanism that gives us two long-running background processes without systemd access: each worker is one line in the component's command list, and Level27 restarts a command's process whenever it stops. Confirmed directly in the panel, the two commands, in full:

periodic (beat embedded, queue `periodic`)

```
cd /var/python/py10102/app && nice -n 10 celery -A website worker -n periodic@%h -Q periodic -c 1
-B \
  --scheduler django_celery_beat.schedulers:DatabaseScheduler \
  --max-tasks-per-child 100 --max-memory-per-child 160000 -l WARNING
```

mailing (default queue `celery`)

```
cd /var/python/py10102/app && nice -n 10 celery -A website worker -n mailing@%h -Q celery -c 1 \
  --max-tasks-per-child 100 --max-memory-per-child 160000 -l INFO
```

Both match the documented worker design (one queue each, concurrency 1, beat only on `periodic`) with three production-only hardening flags not present in the devcontainer's version:

- **`nice -n 10`** — lower CPU priority, since both workers share the machine with gunicorn and Apache.
- **`--max-tasks-per-child 100`** and **`--max-memory-per-child 160000`** (156 MiB) — each worker process is recycled after 100 tasks or once its child exceeds ~156 MB, on top of the whole component's 512 MiB kill limit.
- **`periodic` logs at `WARNING`, `mailing` at `INFO`** — the periodic worker's 10-second dispatcher would flood the log at `INFO`.

> **Why.** The component's 512 MiB ceiling is fixed by the Level27 package, not adjustable from the dashboard; what is a choice is its behaviour at the limit. Set to kill rather than throttle: a stress test with throttle caused a 19-minute mail stall, where kill recovered within seconds of a forced over-limit run.

6 Load testing in production

Two very different things both fall under “load testing” here, with very different risk.

Read-only checks — always safe

- `manage.py capacity_report` over SSH — database size and growth projection.
- `/metrics/` with `METRICS_TOKEN` — the site's own live counters.
- `scripts/deploy.sh --check` — whether the gunicorn concurrency cap is actually in effect.
- A `public`-mode load test (`loadtest/run.sh ... public`) — visitors only, no login, nothing written. Safe directly against the real production site.

A load test that logs in and books — needs the database switch

`loadtest/locustfile.py`'s `mixed` and `rush` modes log in and book real sessions, so against the real production domain they would create fake bookings and send real mail. The script refuses any host that isn't local unless `LOADTEST_CONFIRM_REMOTE_HOST` explicitly names that exact host — a one-time, deliberate opt-in, never a default.

On a demo/showcase deployment that genuinely has no real users yet (confirmed with whoever owns the decision each time, never assumed), `scripts/loadtest_demo.sh` and `scripts/run_demo.sh` automate the only safe way to run these modes against the live domain: repoint the real, running app at the disposable `mysql-loadtest` database for the test window, run the test, then put the real database back — exactly what that component and `mailpit` are provisioned for.

`loadtest_demo.sh switch`, over SSH, never touching the local machine

1. Checks a mail-settings file for Mailpit exists on the server — **refuses without it**, since a test database has made-up addresses.
2. Backs up `~/app/.env` — **refuses to overwrite an existing backup**, so `restore` always has exactly one real `.env` to go back to.
3. Edits only the `DB_*` lines to point at `mysql-loadtest`'s credentials, and the mail settings to Mailpit's; every other setting is untouched.
4. Reloads gunicorn (graceful `HUP`) and warm-restarts both Celery workers, confirming both answer a ping.
5. **Clears the Redis cache (db 0 only)** — the cache still holds the other database's content; on 8 October 2026 skipping this is exactly what let the real site briefly show test data after a restore.
6. **Verifies, not just assumes**: reads `/metrics/` over the Unix socket directly and checks the reported row counts actually match the throwaway database.

`run_demo.sh NAME USERS RATE DURATION [MODE]` then runs `loadtest/run.sh` against the live domain with `LOADTEST_CONFIRM_REMOTE_HOST` set automatically to match — the one place that confirmation is set non-interactively, only because `switch` already made it safe. `restore` reverses all of it and removes the backup file, so a stale backup can never be mistaken for “still switched”. `status` reports which database and mail server are currently live, and whether a backup is present.

A public load test against production, step by step

1. Pick a quiet moment and tell the board; check `/health/` first.

2. Make sure `METRICS_TOKEN` is set in production's `.env`.
3. Start small and stop at the first sign of trouble: 50 users for 3 minutes, then 150 (~30 req/s); don't go past 300 without agreeing it with Level27 first.
4. Watch `/metrics/` while it runs; stop if p95 passes 2s, errors pass 1%, or `/health/` goes to 503.
5. Compare against the devcontainer's own figures and keep the dated results file.

> **Why.** A production run goes through Level27's real proxy with real TLS, so it doubles as an end-to-end check of the proxy's own limits and connection behaviour towards gunicorn, not just the application's own cap.

7 Known constraints and gotchas

Things that only became visible once the app was actually run against Level27's real limits, not the devcontainer's defaults.

- **MySQL's real connection limit is much tighter than the devcontainer's default.** The app's DB user has `max_user_connections = 32` (fixed by the hosting package), against the devcontainer's 151. `gunicorn.conf.py`'s per-worker cap had to be lowered (25 → 8, 8 Oct 2026) once measured against the real limit — the old value produced real `500` errors under a rush, not just graceful `503` refusals.
- **The Celery component's 512 MiB limit is fixed by the package**; its behaviour at the limit (kill, not throttle) is a choice that was made and tested.
- **Redis's eviction policy is `allkeys-lru`, not `volatile-lru`.** The caching design assumes only timed keys (cache entries) are evicted under pressure; under `allkeys-lru` the Celery broker's untimed queue and `/metrics/`'s counters are evictable too. Usage is tiny today, so this is headroom, not an active incident.
- **A Level27 panel change that isn't a code deploy can silently break static/media serving** — the `chmod o+x ~` gotcha (see “The deploy procedure”). The single highest-impact gap documented here: it caused a real, if short, full-site outage already, purely from a panel setting change with no code involved.
- **Several infrastructure questions are still open with Level27**: the proxy's maximum request body size, whether it keeps connections open towards gunicorn, whether MySQL/Redis are dedicated or shared, who rotates the Celery workers' log files, and who patches what.